SOFTWARE PROJECT DOCUMENTATION

PROJECT MANUAL • REV 2.0

Face Verification Attendance Tracker

Full-Stack Web Application Documentation

VERSION	DOCUMENT TYPE	DATE
2.0.0	Technical Manual	April 2026

All rights reserved © 2026

TABLE OF CONTENTS

01 Project Overview 3

02 System Architecture 4

03 Technology Stack 5

04 Project File Structure 6

05 Database Design 7

06 Frontend Documentation 9

07 Backend & Face Verification API 11

08 Face Verification Module 13

09 API Reference 15

10 Authentication & Security 17

11 Real-Time System 19

12 Data Flow Diagrams 20

13 Deployment Guide 22

14 Security Audit & Recommendations 23

15 Future Enhancements 25

SECTION 01

Project Overview

A comprehensive web-based system designed to manage, track, and verify teacher attendance using face recognition and administrative overrides.

6
Database Tables

8
REST API Endpoints

6
Core Modules

Purpose & Goals

The system automates attendance tracking with face-based verification, provides administrators with manual override capabilities, and includes comprehensive reporting features.

Key Features

Camera-Based Check-In
WebRTC capture with face recognition hook for identity verification.

Live Dashboard + Compare.php
Statistics auto-refresh every 15s; face verification page with match scoring and admin overrides.

Monthly Reports
Per-teacher attendance rates with visual progress bars.

Role-Based Access + Override
Admins can manually verify/override face recognition mismatches with notes.

SECTION 02

System Architecture

SSR + REST API hybrid architecture with face verification as a core module.

Layer Overview

Layer	Technology	Responsibility
Presentation	PHP templates, Bootstrap 5	Render HTML pages
Face Verification UI	compare.php	Admin verification interface with match scoring
Application	PHP 7.x/8.x	Business logic, authentication, file handling
Face Verification API	face_verification_api.php	All verification operations (get_attendances, override, etc.)
Data	MySQL / MariaDB	Persistent storage with 6 tables

SECTION 03

Technology Stack

Frontend

Technology	Role
Vanilla JavaScript (ES6)	DOM, Fetch API, camera, canvas
Bootstrap 5	Responsive layout, UI components
HTML5 Canvas / WebRTC	Camera frame capture as image blob

Backend

Technology	Role
PHP 7.4+ / 8.x	Server-side scripting, API endpoints
MySQLi Extension	Prepared statements, transactions
password_hash()	bcrypt password hashing

SECTION 04

Project File Structure

```
/project ├── /api │ └── face_verification_api.php # All verification operations
├── compare.php # Admin verification interface ├── dashboard.php # Live
statistics ├── attendance.php # Webcam check-in ├── teachers.php # Teacher
management ├── reports.php # Monthly reports └── /uploads ├── /teachers #
Teacher profile photos └── /attendance # Check-in snapshots
```

SECTION 05

Database Design

Six relational tables with foreign key constraints ensure data integrity.

Table: attendance

Column	Type	Description
`id`	INT PK	Primary key
`teacher_id`	INT FK	References teachers(id) ON DELETE CASCADE
`date`	DATE	Attendance date
`check_in`	TIME	Check-in time
`check_out`	TIME	Check-out time (fully supported in compare.php)
`status`	VARCHAR(50)	'Present', 'Late', 'Absent'
`captured_image`	VARCHAR(255)	Path to captured face image

Table: face_verification_logs

Column	Type	Description
`id`	INT PK	Verification record ID
`attendance_id`	INT FK	References attendance(id) ON DELETE CASCADE
`teacher_id`	INT FK	References teachers(id)
`match_score`	FLOAT	Face matching confidence (0-100)
`admin_override`	TINYINT	1 if admin manually overrode

Column	Type	Description
`admin_notes`	TEXT	Optional notes from admin
`verified_by`	INT FK	References users(id) for audit trail

Table: admin_verifications

Stores email verification codes for admin account creation.

Column	Type	Description
`verification_code`	VARCHAR(50) UNIQUE	Temporary code
`expires_at`	DATETIME	Code expiration timestamp

SECTION 06

Frontend Documentation

compare.php – Face Verification Interface

The admin verification page includes:

- **Filtering** — By verification status (unverified/verified/overridden), attendance status (present/absent/late), teacher, and date.
- **Statistics Cards** — Total records, needs verification, verified, admin overridden.
- **Attendance Table** — Displays **Date, Teacher, Check In, Check Out, Duration, Attendance Status, Match Score, Verification Status, Verified By, Actions.**
- **Verification Modal** — Shows captured face vs teacher database photo, match score, current status, admin notes, and override buttons.

Check-Out and Duration Features
The attendance table now includes a **Check Out** column and calculates the **Duration** (e.g., "8h 30m") between check-in and check-out. The verification modal also displays check-out time for reference.

JavaScript Functions in compare.php

Function	Purpose
`loadUnverifiedAttendances()`	Fetches attendance records with current filters
`displayAttendanceList()`	Renders table with check-out and duration columns
`calculateDuration(checkIn, checkOut)`	Computes duration between check-in and check-out times
`openVerificationModal()`	Loads verification details into modal

Function	Purpose
`overrideAttendance(confirmMatch)`	Submits admin verification decision (match/not match)

SECTION 07

Face Verification API

A comprehensive JSON API handling all face verification operations.

API Endpoints (face_verification_api.php)

Action	Method	Description
`get_teachers`	GET	Returns all teachers for filter dropdown
`get_attendances`	GET	Paginated attendance records with filters, includes check_out and duration calculation support
`get_verification_details`	GET	Returns captured image, teacher photo, match score, and check_out for modal
`override_attendance`	POST	Admin override with notes; updates attendance status based on decision

Face Comparison Algorithm

The API implements a three-method weighted scoring system:

- **Perceptual Hash (40%)** — Generates 64-bit hash, calculates Hamming distance.
- **Histogram Comparison (30%)** — 48-bin RGB histogram with Bhattacharyya coefficient.
- **Structural Similarity (30%)** — SSIM index on resized 100×100 images.

Final score: weighted average, boosted if hash and histogram scores are high, penalized if structural similarity is low.

SECTION 08

Face Verification Workflow

Attendance captured

When a teacher checks in, captured_image saved and face_verification_logs record created with initial match_score.

Admin opens compare.php

Filters display unverified records (match_score < 70). Each row shows match score, verification status.

Admin verifies

Clicks "Verify" button, modal shows side-by-side face comparison with match score.

Admin decision

Chooses "Confirm Match (Present)" or "Mark as Not Matching (Absent)". Optionally adds notes.

Database updated

face_verification_logs updated with admin_override=1, notes, verified_by. Attendance status updated if needed.

SECTION 09

API Reference

GET get_attendances (with check_out support)

```
// Request
GET api/face_verification_api.php?action=get_attendances&status=unverified&teacher_

// Response includes check_out and calculated duration
{
  "attendances": [{
    "id": 101,
    "check_in": "08:15",
    "check_out": "16:45",
    "duration": "8h 30m",
    "match_score": 65.2,
    "attendance_status": "Present"
  }]
}
```

POST override_attendance

```
// Request (multipart/form-data)
action: override_attendance
attendance_id: 101
teacher_id: 5
confirm_match: 1
admin_notes: "Face matches teacher photo. Verified."

// Response
{
  "success": true,
  "message": "Attendance confirmed as matching. Teacher marked as Present",
  "attendance_status": "Present",
```

```
    "verification_status": "overridden"
  }
```

SECTION 10

Authentication & Security

✓ **Prepared Statements**

All queries in face_verification_api.php use bind_param() for SQL injection prevention.

✓ **Role-Based Access**

override_attendance action checks admin role before allowing changes.

✓ **Admin Notes Validation**

Requires notes when marking as not matching, mandatory for audit trail.

△ **Face recognition algorithm**

Uses deterministic algorithms only; no external API calls, but not neural-network accurate.

SECTION 11

Real-Time System

Polling Implementation

- **dashboard.php** — Polls every 15 seconds for stats counts.
- **compare.php** — Polls every 30 seconds for updated records (setInterval).
- **Manual refresh** — "Refresh" button and filter changes trigger immediate load.

SECTION 12

Data Flow Diagrams

Face Verification Flow

Admin filters records

GET request to get_attendances with status=unverified

Select record to verify

get_verification_details loads images, match_score, check_out

Admin submits decision

POST override_attendance with confirm_match, notes

Transaction updates

face_verification_logs SET admin_override=1, notes, verified_by; attendance status updated if confirm_match=0 sets to 'Absent'

SECTION 13

Deployment Guide

Requirements

- PHP 7.4+ with MySQLi, GD extension (for image hashing/histograms)
- MySQL 5.7+ with InnoDB support for transactions
- Write permissions on `/uploads/teachers/` and `/uploads/attendance/`

Database Initialization

```
mysql -u root -p < database/schema.sql

-- Insert default system settings
INSERT INTO system_settings (id, system_name, sidebar_color)
VALUES (1, 'Teacher Attendance System', '#2c3e50');
```

Face Directory Permissions
The face comparison functions require temporary file access. Ensure `/tmp` is writable for uploaded face images during comparison.

SECTION 14

Security Audit & Recommendations

7	2	2
Controls Implemented	Warnings	Critical Risks

Prioritized Recommendations

- **CSRF Protection —** Add tokens to POST endpoints.
- **Rate Limiting —** Implement on login and API endpoints.
- **Upload Validation —** Add MIME type checking for face images.
- **Session Regeneration —** Call session_regenerate_id(true) after login.

SECTION 15

Future Enhancements

Immediate Priorities

- **Real Face Recognition —** Replace deterministic algorithms with face-api.js or AWS Rekognition.
- **Bulk Verification —** Already partially implemented in API, add UI to compare.php for batch operations.
- **Check-Out Enforcement —** Add check-out button to attendance.php; log departure time with face re-verification.
- **Email Alerts —** Notify admin of low match scores requiring verification.

Check-Out Enhancement Roadmap
- Add "Check Out" button on attendance.php for logged-in teacher.
- Record check_out timestamp in attendance table.
- Optionally require face re-verification on check-out for high security.
- Extend compare.php to show matching verification status for both check-in and check-out events.